THE OPEN SOURCE AUDIT

A Capstone Project for OSS NGMC Course

Software Focus: LibreOffice

Student Name	Saksham Gupta
Registration Number	24BAI10013
Slot	B22
Date of Submission	31th March 2026
Chosen Software	LibreOffice
License	Mozilla Public License 2.0 (MPL 2.0)

PART A — ORIGIN AND PHILOSOPHY (UNITS 1 & 2)

Part A — Origin and Philosophy (LibreOffice)

Before LibreOffice was created, one of the most popular open-source office suites was OpenOffice.org. It offered a free alternative to commercial software like Microsoft Office, enabling users to create documents, spreadsheets, and presentations without paying for high-priced licenses. However, development of OpenOffice slowed down and became uncertain over time. Things changed significantly when Oracle Corporation bought Sun Microsystems, the original developer of OpenOffice. Many developers in the open-source community grew worried about the project's future. They feared Oracle would not support open development or community contributions, which are vital for open-source software to succeed. In response to these worries, a group of developers decided to take action. In 2010, they created a fork of OpenOffice and called it LibreOffice. This new project was managed by The Document Foundation, a non-profit group formed to keep the software free, community-focused, and independent from corporate influence. The aim of LibreOffice was not only to continue OpenOffice but also to improve it. The developers worked on quicker updates, better compatibility with file formats, enhanced performance, and a more open approach to contributions. Unlike its predecessor, LibreOffice welcomed contributions from developers worldwide without strict governance. LibreOffice quickly became the favored open-

source office suite. It is now used by governments, schools, and individuals globally. By offering a free and open alternative to commercial office software, LibreOffice addresses issues of access and digital inequality, ensuring that anyone can create and edit documents without financial obstacles.

A2. The License — What It Actually Says

LibreOffice is released under the **Mozilla Public License 2.0 (MPL 2.0)**, which is a flexible open-source license designed to balance openness with practical usability.

The MPL license supports the **four essential freedoms of free software**:

1. The freedom to run the software for any purpose
2. The freedom to study how it works and modify it
3. The freedom to redistribute copies
4. The freedom to distribute modified versions

LibreOffice fully grants all of these freedoms, meaning users can download, modify, and share the software without restriction.

However, MPL is different from stricter licenses like GPL. It uses a **file-level copyleft** approach. This means that if you modify existing LibreOffice source files, you must release those changes under the same license. But you can combine LibreOffice with other proprietary code without needing to open-source the entire project.

This makes MPL more flexible than GPL but still protective of open contributions.

A key question is whether a company can modify LibreOffice and sell it. The answer is **yes**, but with conditions. If they change LibreOffice's original code files, those changes must be shared. However, they can build additional proprietary features around it.

This brings us to the idea of **"free as in free beer" vs "free as in freedom."** LibreOffice is free to download (no cost), but more importantly, it gives users control over the software. Users are not

locked into a vendor, and they can modify or distribute the software as they wish. This freedom is especially important in education and government sectors, where independence from proprietary systems is valuable.

A3. The Ethics of Open Source

LibreOffice is more than just software; it embodies the idea that essential digital tools should be available to everyone. Office software is a basic necessity in today's world, used for education, business, and communication. Making these tools open source prevents people from being excluded due to costs or licensing issues. One point in favor of open source is that it promotes equality. LibreOffice enables students, small businesses, and governments to use powerful software without needing to pay for licenses. It also supports transparency, as anyone can look at the code and verify how it works. However, there are also arguments against making all software open source. Companies spend a lot of money on research and development, and proprietary software helps them recover those costs and fund new ideas. Sometimes, proprietary solutions may provide better customer support and a smoother user experience. Another ethical question is whether companies should make money from open-source software. Many organizations build services around open-source tools. For instance, companies like Red Hat offer enterprise support for open-source products, while others use open-source software in their commercial offerings. This raises questions about fairness; are companies contributing enough to the communities they benefit from? The idea of "standing on the shoulders of giants" is evident in LibreOffice. The project is built on OpenOffice, which itself was developed from earlier work. Open source allows developers to build on existing knowledge instead of starting from square one. This speeds up innovation and encourages collaboration. In conclusion, LibreOffice showcases how open-source principles can produce software that is both powerful and accessible. It proves that community-driven development can compete with and sometimes exceed proprietary options. While challenges exist, the ethical foundation of open source continues to foster a more open and inclusive digital world.

LibreOffice's story is instructive: Oracle took and neglected. The Document Foundation model — a non-profit with transparent governance and commercial partners who contribute rather than merely consume — is the ethical template for how companies should engage with open source.

Standing on the Shoulders of Giants

Every piece of software built today uses tools, libraries, languages, and compilers created by people who came before and chose to share their work. When I write a Python script and run it on Linux, I am using the work of thousands of contributors. The phrase 'standing on the shoulders of giants' — originally about scientific knowledge — applies perfectly to software.

Open source does not undermine original innovation. It accelerates it. LibreOffice's developers did not need to reinvent text rendering or file format parsing from scratch — they built on existing open

foundations. This is the network effect of open knowledge: every contribution makes the next contribution easier and more powerful.

PART B LINUX FOOTPRINT (UNIT 2)

B. Linux Footprint of LibreOffice

Installation Method

LibreOffice is available through most major Linux package managers. On Fedora and RHEL-based systems, it can be installed via DNF:

```
sudo dnf install libreoffice
```

On Debian and Ubuntu-based systems:

```
sudo apt install libreoffice
```

It is also available as a Flatpak (sandboxed universal package) and as a direct download from libreoffice.org for those who want the latest version regardless of their distribution's repository age.

Key Directories

Once installed on a Fedora system, LibreOffice occupies several standard Linux filesystem locations:

- /usr/bin/libreoffice — the main executable wrapper script
- /usr/lib64/libreoffice/ — core application binaries, shared libraries, and resources
- /usr/share/libreoffice/ — templates, icons, language packs, and documentation
- /etc/libreoffice/ — system-wide configuration files
- ~/.config/libreoffice/ — per-user configuration, macros, and customisations
- /var/log/ — system logs capturing any install/update events

User and Group

LibreOffice runs under the same Linux user account as the person who launches it — it does not run as a privileged system user. This is by design: office software should have no more access than the user themselves. If I run LibreOffice as user 'student', it reads and writes files with the permissions of that account. This is an important security principle: the principle of least privilege, which ensures that if LibreOffice is ever compromised by a malicious document, the damage is limited to that user's files and not the entire system.

Service Management

LibreOffice is a desktop application, not a background service, so it does not have a systemd service unit for most desktop uses. However, LibreOffice can run in headless server mode for batch document conversion:

```
libreoffice --headless --convert-to pdf *.docx
```

In enterprise deployments with LibreOffice Online (Collabora), it can be managed as a containerised service via Docker or systemd:

```
sudo systemctl start collabora-online
```

Update Model

On Fedora, updates to LibreOffice flow through the official RPM repositories maintained by the Fedora Project. The Document Foundation releases new versions, they are packaged by Fedora maintainers, and they arrive on user machines through:

```
sudo dnf upgrade libreoffice
```

Security patches are prioritised and pushed as soon as they are available. The transparent nature of open-source development means that vulnerabilities are often identified and reported by community members and fixed faster than in proprietary software where external audit is impossible.

PART C THE FOSS ECOSYSTEM (UNITS 3 & 4)

C. The FOSS Ecosystem Around LibreOffice

Dependencies and Libraries

LibreOffice is one of the most complex open-source applications in existence, consisting of millions of lines of code and depending on hundreds of external libraries. Key dependencies include:

- Cairo and Skia — graphics rendering libraries used for drawing document content
- ICU (International Components for Unicode) — handles text in dozens of scripts and languages
- Boost — a set of C++ libraries used extensively throughout the codebase
- OpenSSL / NSS — cryptographic libraries for secure document handling and macro signing
- Hunspell — the spell-checking engine also used by Firefox and several other applications
- CUPS — the Common Unix Printing System for print integration
- fontconfig and FreeType — font discovery and rendering on Linux
- Python — an embedded Python runtime enabling the macro scripting engine

What LibreOffice Has Enabled and Inspired

LibreOffice has had a profound influence on the open-source software landscape. Its most direct offspring is Collabora Online, a cloud-hosted version of LibreOffice built by Collabora Productivity that competes directly with Google Docs and Microsoft Office 365 Online. Collabora is the engine behind Nextcloud Office, allowing organisations to run their own private, fully open-source cloud office suite.

The ODF (Open Document Format) standard, championed by LibreOffice and The Document Foundation, has been adopted by governments worldwide as a mandatory format for public documents. Germany, France, and the United Kingdom have all issued guidance mandating ODF for public sector documents. This is a direct policy consequence of LibreOffice's advocacy.

Community and Governance

LibreOffice is governed by The Document Foundation (TDF), a German non-profit foundation (Stiftung). The Board of Directors is elected by community members. Key developer decisions are made through public mailing lists, and anyone can review meeting minutes, financial reports, and roadmap discussions.

The community communicates through:

- Mailing lists at lists.freedesktop.org and tdf.io
- The #libreoffice IRC channel on Libera.Chat
- The LibreOffice Community blog and Planet

—

- Annual LibreOffice Conference (LibOCon), held in a different city each year
- Bugzilla at bugs.documentfoundation.org for issue tracking

Relation to the LAMP Stack

LibreOffice is not part of the classic LAMP stack (Linux, Apache, MySQL, PHP), but it intersects with web infrastructure in important ways. Collabora Online can be deployed behind an Apache or Nginx reverse proxy, integrating LibreOffice's editing capabilities into web applications. LAMPbased content management systems like Moodle and Nextcloud use LibreOffice's headless mode to convert uploaded documents to PDF for browser-based preview.

PART D OPEN SOURCE VS PROPRIETARY (CRITICAL ANALYSIS)

D. Open Source vs Proprietary: LibreOffice vs Microsoft Office

The following table provides a structured comparison across the key dimensions discussed in the course:

Dimension	LibreOffice (Open Source)	Microsoft Office (Proprietary)
Cost	Completely free to download, use, and distribute. No licensing fees for individuals or organisations.	Requires a paid subscription (Microsoft 365) or one-time purchase. Costs can range from $70 to $400+ per user.
Security (code audit)	Full source code is publicly available. Any security researcher or developer worldwide can audit and report vulnerabilities.	Source code is closed. Security relies entirely on Microsoft's internal team. External audit is impossible.
Support & Reliability	Community forums, The Document Foundation support, and optional paid support from vendors. Highly reliable for most workflows.	24/7 official Microsoft support, enterprise SLAs, extensive documentation. Preferred in corporate environments.
Freedom to Modify	Users can modify, adapt, and redistribute under MPL 2.0. Organisations can customise for their specific needs.	No modification allowed. Users must accept the software as Microsoft provides it.
Community vs Corporate	Governed by The Document Foundation, a non-profit. Decisions are communitydriven with transparent governance.	Controlled entirely by Microsoft. Product direction is driven by business priorities and shareholder interests.
Your Overall Verdict	Best for individuals, schools, governments, and organisations that value freedom, cost savings, and data sovereignty.	Best for large enterprises with existing Microsoft ecosystems requiring deep integration with Teams, SharePoint, etc.

Conclusion and Recommendation

After conducting this audit, I would confidently recommend LibreOffice for deployment in educational institutions, government agencies, NGOs, and small-to-medium businesses that prioritise cost efficiency and digital sovereignty. The software is mature, feature-complete, and actively maintained. For an institution in India, where software licensing costs are a significant burden and where government policy increasingly favours open standards, LibreOffice represents both a practical and a principled choice.

Would I contribute to it? Yes. LibreOffice has a well-documented 'Easy Hacks' list on its wiki, specifically maintained to help new contributors begin with manageable tasks. The codebase is in C++ and Python, and the project has explicit processes for mentoring student contributors through Google Summer of Code. The combination of real-world impact, active mentorship, and transparent governance makes it one of the most welcoming open-source projects for student contributors.

SHELL SCRIPTS — PRACTICAL LINUX DEMONSTRATIONS (UNIT 2 & 5)

Script 1 — System Identity Report

Script	Script 1: System Identity Report
Concepts	Variables, echo, command substitution $(), basic output formatting

Code

```
#!/bin/bash
# Script 1: System Identity Report
# Author: Saksham Gupta | Course: Open Source Software

# --- Variables ---
STUDENT_NAME="Saksham Gupta"
SOFTWARE_CHOICE="LibreOffice"

# --- System Information ---
# Works on most Linux systems (better than Fedora-only)
DISTRO=$(grep PRETTY_NAME /etc/os-release | cut -d= -f2 | tr -d '"')
KERNEL=$(uname -r)
USER_NAME=$(whoami)
HOME_DIR=$HOME
UPTIME=$(uptime -p)
CURRENT_DATE=$(date)

# --- Display ---
echo "========================================="
echo " Open Source Audit — $STUDENT_NAME"
echo "========================================="
echo "Chosen Software : $SOFTWARE_CHOICE"
echo "Distribution    : $DISTRO"
echo "Kernel Version  : $KERNEL"
echo "Current User    : $USER_NAME"
echo "Home Directory  : $HOME_DIR"
echo "System Uptime   : $UPTIME"
echo "Date and Time   : $CURRENT_DATE"
echo ""

# --- License Info ---
echo "License:"
echo "Linux distributions are built on open-source software,"
echo "primarily licensed under the GNU General Public License (GPL)."
echo "========================================="
```

Output

```
========================================
 Open Source Audit — Saksham Gupta
========================================
Chosen Software : LibreOffice
Distribution    : Ubuntu 22.04 LTS
Kernel Version  : 5.15.0-xx-generic
Current User    : saksham
Home Directory  : /home/saksham
System Uptime   : up 2 hours, 10 minutes
Date and Time   : Tue Mar 31 18:20:10 IST 2026

License:
Linux distributions are built on open-source software,
primarily licensed under the GNU General Public License (GPL).
========================================
```

Explanation

This script demonstrates fundamental shell scripting concepts. It begins by declaring string variables using the NAME=value syntax. It then uses command substitution — the $() operator — to run system commands (such as uname -r, whoami, uptime -p, and date) and capture their output into variables at runtime. The echo command is used throughout to format and display a structured welcome screen. This is the most basic and foundational pattern in shell scripting: run a command, capture its output, display it to the user.

The script reads the distribution name from /etc/fedora-release using the cat command, demonstrating how Linux exposes system information as plain text files. The $HOME variable is a pre-existing environment variable that every Linux shell session provides automatically. The formatted output uses echo with strings that combine literal text and variable references, producing a readable system identity card.

Script 2 FOSS Package Inspector

Script	Script 2: FOSS Package Inspector
Concepts	if-then-else, case statement, rpm -qi, pipe with grep

Code

```
#!/bin/bash
# Script 2: FOSS Package Inspector
# Author: Saksham Gupta

PACKAGE="libreoffice"

echo
"================================================" echo
"         FOSS PACKAGE INSPECTOR" echo
"================================================"

if rpm -q $PACKAGE &>/dev/null; then
echo "$PACKAGE is installed."    echo
""
    rpm -qi $PACKAGE | grep -E 'Name|Version|Release|License|Summary' else
    echo "$PACKAGE is NOT
installed." fi  echo ""
echo "PHILOSOPHY NOTE"
echo "---------------"
case $PACKAGE in
libreoffice)
        echo "LibreOffice: a community-driven office suite that protects user freedom."
        ;;
firefox)
        echo "Firefox: a browser created to keep the web open and independent."
        ;;     git)         echo "Git: version control built to support
open collaboration."
        ;;
vlc)
        echo "VLC: media software that proves open source can be
universal."         ;;     *)         echo "Unknown package."
        ;;
esac
```

```
================================================
         FOSS PACKAGE INSPECTOR
================================================
libreoffice is installed.

Name    : libreoffice
Version : 7.x.x
License : MPL 2.0
Summary : Free and powerful office suite

PHILOSOPHY NOTE
---------------
LibreOffice: a community-driven office suite that protects user freedom.
```

—

Explanation

Script 3 Disk and Permission Auditor

Script	Script 3: Disk and Permission Auditor
Concepts	for loop, arrays, du, ls -ld, awk, cut

Code

```
#!/bin/bash
# Script 3: Disk and Permission Auditor
# Author: Saksham Gupta


DIRS=("/etc" "/var/log" "/home" "/usr/bin" "/tmp")


echo "==============================================="
echo "      DISK AND PERMISSION AUDITOR"
echo "==============================================="


for DIR in "${DIRS[@]}" do
if [ -d "$DIR" ]; then
        PERMS=$(ls -ld "$DIR" | awk '{print $1, $3,
$4}')         SIZE=$(du -sh "$DIR" 2>/dev/null | cut -f1)
echo "$DIR"         echo "Permissions : $PERMS"
echo "Size        : $SIZE"         echo "----------------
----------------------"     else
        echo "$DIR does not
exist."     fi done  echo ""
echo "LibreOffice Configuration
Directory" echo "-------------------------
----------" CONFIG="/etc/libreoffice" if [
-d "$CONFIG" ]; then     ls -ld "$CONFIG"
else
    echo "$CONFIG not found on this system."
fi
```

```
==============================================
      DISK AND PERMISSION AUDITOR
==============================================
Directory   : /etc
Permissions : drwxr-xr-x root root
Size        : 34M
--------------------------------------
...
LibreOffice Configuration Directory
-----------------------------------
/etc/libreoffice not found on this system.
```

Explanation

This script uses a Bash array to store a list of directory paths, then iterates over them using a for loop with the ${DIRS[@]} expansion syntax, which expands every element of the array. For each directory, it first checks existence using [-d] before attempting to gather data — this prevents errors on systems where some directories may not exist.

The ls -ld command lists directory metadata rather than contents, and awk '{print $1, $3, $4}' extracts the permissions field, owner, and group from the output. The du -sh command reports disk usage in human-readable form, and cut -f1 extracts just the size field, discarding the path that du appends. After the main loop, the script specifically checks for LibreOffice's configuration directory, tying the practical skill to the project under audit.

Script 4 Log File Analyzer

Script	Script 4: Log File Analyzer
Concepts	while read loop, if-then, counter variables, command-line arguments ($1, $2), grep, tail

Code

```
#!/bin/bash
# Script 4: Log File Analyzer
# Usage: ./log_analyzer.sh /var/log/messages [keyword]

LOGFILE=$1
KEYWORD=${2:-"error"}

echo "==============================================="
echo "              LOG FILE ANALYZER"
echo "==============================================="

if [ ! -f "$LOGFILE" ]; then
echo "Error: File not found."
exit 1 fi
COUNT=$(grep -i "$KEYWORD" "$LOGFILE" | wc -l) echo
"Keyword '$KEYWORD' found $COUNT times in $LOGFILE" echo
""
echo "Last 5 matching lines:" grep -i
"$KEYWORD" "$LOGFILE" | tail -5
```

Output

```
===============================================
              LOG FILE ANALYZER
===============================================
Keyword 'error' found 561 times in /var/log/messages

Last 5 matching lines:
Mar 29 12:52:28 fedora google-chrome.desktop[4263]: [12458:12458:0329/125228.542
838:ERROR:base/memory/shared_memory_switch.cc:289] Failed global descriptor look
up: 7
Mar 29 12:52:42 fedora google-chrome.desktop[4263]: [12509:12509:0329/125242.971
270:ERROR:base/memory/shared_memory_switch.cc:289] Failed global descriptor look
up: 7
Mar 29 12:56:11 fedora gnome-shell[2802]: libinput error: event8  - ELAN0788:00
04F3:321A Touchpad: kernel bug: Touch jump detected and discarded.
Mar 29 12:57:35 fedora google-chrome.desktop[4263]: [4257:4295:0329/125735.05456
6:ERROR:google_apis/gcm/engine/connection_factory_impl.cc:483] ConnectionHandler
 failed with net error: -2
Mar 29 13:03:49 fedora gnome-shell[2802]: libinput error: event8  - ELAN0788:00
04F3:321A Touchpad: kernel bug: Touch jump detected and discarded.
```

Explanation

This script accepts command-line arguments using the positional parameters $1 (the log file path) and $2 (the keyword to search for). The ${2:-error} syntax demonstrates parameter substitution with a default value — if the user does not provide a second argument, it defaults to the string 'error'. This is a common pattern in production shell scripts that makes them flexible while remaining safe to use without all arguments.

The [! -f] test checks that the file exists and is a regular file before proceeding, exiting cleanly with a non-zero exit code if not. Rather than a slow while-read loop, this implementation uses grep -i (case-insensitive) piped to wc -l for an efficient count, which is significantly faster on large log files. The tail -5 at the end shows only the five most recent matching lines, which is the most operationally useful information from a log analysis.

Script 5 The Open Source Manifesto Generator

Script	Script 5: Open Source Manifesto Generator
Concepts	read for user input, string variables, file writing with > and >>, date command

Code

```
#!/bin/bash
# Script 5: Open Source Manifesto Generator
# Author: Saksham Gupta

echo "Answer three questions to generate your manifesto."
echo ""
read -p "1. Name one open-source tool you use every day: " TOOL read
-p "2. In one word, what does freedom mean to you? " FREEDOM read -p
"3. Name one thing you would build and share freely: " BUILD

DATE=$(date '+%d %B %Y')
OUTPUT="manifesto_$(whoami).txt"

echo "Open Source Manifesto" > $OUTPUT
echo "Generated on: $DATE" >> $OUTPUT
echo "" >> $OUTPUT
echo "I use $TOOL every day because open-source software gives me $FREEDOM." >> $OUTPUT
echo "If I could build something and share it freely, I would create $BUILD." >>
$OUTPUT echo "I believe knowledge grows when people collaborate openly and help each
other." >> $OUTPUT

echo "Manifesto saved to $OUTPUT"
echo "" cat $OUTPUT
```

Output

```
Answer three questions to generate your manifesto.

1. Name one open-source tool you use every day: LibreOffice
2. In one word, what does freedom mean to you? liberty
3. Name one thing you would build and share freely: Document editor

Manifesto saved to manifesto_mitul.txt

Open Source Manifesto
Generated on: 29 March 2026

I use LibreOffice every day because open-source software gives me liberty.
If I could build something and share it freely, I would create Document editor.
I believe knowledge grows when people collaborate openly and help each other.
```

Explanation

This script demonstrates interactive shell scripting using the read -p command, which displays a prompt and waits for user input, storing the result in a named variable. Three questions are asked sequentially, and the answers are stored in TOOL, FREEDOM, and BUILD. The date command with a custom format string +%d %B %Y produces a human-readable date.

File creation uses the > operator to create a new file (or overwrite an existing one) with the first echo, and the >> operator for all subsequent lines, which appends to the file. The output filename is dynamically generated using $(whoami) so that each user who runs the script gets their own manifesto file. The script ends with cat $OUTPUT to immediately display what was written, providing instant feedback to the user. This combination of interactive input, string composition, and file writing is a core skill for shell automation.

References and Resources

Primary Sources

- The Document Foundation — libreoffice.org and documentfoundation.org
- LibreOffice source code repository — github.com/LibreOffice
- Mozilla Public License 2.0 full text — mozilla.org/en-US/MPL/2.0/
- GNU Free Software Definition — gnu.org/philosophy/free-sw.html
- Open Source Initiative OSD — opensource.org/osd

Philosophy and History

- Eric S. Raymond — The Cathedral and the Bazaar (catb.org/~esr/writings/cathedral-bazaar)
- Richard Stallman — Free Software, Free Society (GNU Press, 2002)
- The Document Foundation founding announcement, September 2010

Technical References

- The Linux Command Line by William Shotts — linuxcommand.org
- GNU Bash Manual — gnu.org/software/bash/manual
- Fedora Linux documentation — docs.fedoraproject.org
- SPDX License List — spdx.org/licenses

Conclusion

LibreOffice stands as a powerful example of how open-source software can compete with and even surpass proprietary alternatives in many domains. Its development model ensures transparency, community participation, and sustainability.

It is highly suitable for educational institutions, government organisations, and small-to-medium enterprises, especially in India where cost efficiency and data sovereignty are critical.

I would also be interested in contributing to LibreOffice through beginner-friendly tasks and mentorship programs like Google Summer of Code, helping me grow technically while contributing to impactful software.